Discovery Executive Summary

Project: discovery-test · **Generated:** 11/08/2026, 16:16:22

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Code Quality & Complexity Analysis	—

---

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo combining a Laravel 12 PHP backend, a React 19 TypeScript SPA frontend, and a Node.js/Express dev-API shim — all serving a voice observability product with two business domains: **Discovery** (IVR mapping) and **Connect** (TFN reachability monitoring). The overall architecture is immature: controllers perform direct Eloquent ORM access, embed business logic (reachability calculations, IVR tree building), and duplicate code across files, while no Repository layer exists. The most severe hotspots are a fat `LegacyReportController` combining DB queries, KPI math, and response formatting; a `buildTree()` function copy-pasted verbatim into two controllers; and a reachability formula that appears three times (two PHP controllers, one service). On the frontend, `LegacyMonitorPoller.jsx` is a class component with an interval memory leak, and `LegacyDashboardWidget.tsx` mixes multi-domain data fetching with rendering, bypassing the TanStack Query pattern used elsewhere. The dominant systemic risk is **change amplification**: any change to the reachability threshold (currently hardcoded at 90%) or IVR tree shape must be applied in multiple controllers, a service, and a Node.js route handler — with no test coverage to catch a missed update.

§1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~74 LOC avg	Good
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	13 direct Eloquent calls across 4 controllers	Moderate
H3	Missing Repository Pattern	Direct DB access outside repos	<10	10–20	>20	23+ Eloquent static calls; 0 repositories exist	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files with business logic	0	1–5	>5	1 (`LegacyDataMapper` using unsafe `extract()`)	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% — all queries via Eloquent	Good
H7	God Classes	Files >1000 LOC	0	1–3	>3	0	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	6 (LegacyReportController, RealTimeTestService, DashboardController)	High Risk
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	0%	Good
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	~120 LOC avg	Good

F2	Missing Frontend Service Layer	Components with inline API calls	<10	10–20	>20	6 components with direct `api.get/post`	Moderate
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	Good
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	1 level	Good
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (`LegacyMonitorPoller.jsx` , `LegacyDashboardWidget.tsx`)	Moderate
H10	Duplicate Business Logic *(additional)*	Copy-pasted algorithm occurrences	0	1–2	>2	3 (`buildTree()` ×2 controllers + JS; reachability ×2 controllers + service)	High Risk

§1.4 Actions Required

Hotspot	Action	Rating	Priority
H3 Missing Repository Pattern	Create `DiscoveryJobRepository` and `ConnectMonitorRepository` ; bind interfaces in `AppServiceProvider` ; update `RealTimeTestService` for constructor DI	High Risk	Critical
H8 Domain Boundary Violations	Split `RealTimeTestService` into domain-specific orchestrators; route `LegacyReportController` cross-domain reads through `DiscoveryService` ; create `DashboardService`	High Risk	Critical
H10 Duplicate Business Logic	Extract `buildTree()` into `DiscoveryService` ; extract reachability formula into `ConnectService::computeReachability()` ; delete private controller copies	High Risk	Critical
H2 Missing Service Layer	Create `ConnectService` and `DashboardService` ; thin controllers to single-line service delegation	Moderate	High
H5 Shared Utility Abuse	Delete `LegacyDataMapper.php` ; replace `extract()` calls with explicit destructuring; add Form Request validation	Moderate	High
F5 Legacy Component Patterns	Convert `LegacyMonitorPoller.jsx` to TypeScript function component with `useEffect` cleanup; add `ErrorBoundary.tsx`	Moderate	High
F2 Missing Frontend Service Layer	Create `connectApi.ts` and `discoveryApi.ts` ; migrate `LegacyDashboardWidget.tsx` to `useQuery` ; centralize all endpoint paths	Moderate	Medium

§1.5 Expected Outcomes

- **Testability:** Repository interfaces allow `RealTimeTestService` , `ConnectController` , and `DashboardController` to be unit-tested with in-memory fakes instead of live databases; the reachability formula and IVR tree builder gain straightforward coverage.
- **Change isolation:** Splitting `RealTimeTestService` by bounded context means a Discovery schema change does not require touching any Connect file, shrinking blast radius for routine feature work.
- **Single source of truth:** Extracting `buildTree()` and the reachability formula into dedicated service methods eliminates three-location duplication; the 90% alert threshold becomes a named constant changed in exactly one place.
- **Frontend reliability:** Adding `ErrorBoundary.tsx` and fixing the `LegacyMonitorPoller` interval leak prevents full-page crashes on API failure and stops memory accumulation during navigation.
- **Architectural readiness:** Bounded-context separation with ACLs between Discovery and Connect prepares the platform for eventual microservice extraction — either module can be independently deployed once its data access is fully encapsulated behind a service interface.

The full report (34 KB, including all code evidence and Mermaid diagrams) is saved to `docs/discovery/01-architecture-design.md` . The orchestration UI will convert it to PDF automatically.","stop_reason":"end_turn","session_id":"7a812e19-13a2-40bc-8fd8-11d9e79989f8","total_cost_usd":1.8304448000000004,"usage":{"input_tokens":5,"cache_creation_input_tokens":28476,"cache_read_input_tokens":99390,"output_tokens":4450,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":28476,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":2306,"cache_read_input_tokens":43454,"cache_creation_input_tokens":2135,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":2135},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":10502,"outputTokens":15,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.010577,"contextWindow":200000,"maxOutputTokens":32000},"claude-sonnet-4-6":{"inputTokens":46,"outputTokens":35734,"cacheReadInputTokens":2440596,"cacheCreationInputTokens":129992,"webSearchRequests":0,"costUSD":1.8198678000000004,"contextWindow":200000,"ma[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"3f79d4f6-72b5-419b-a4c6-dfbf21bb9356"}

---

2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

The Klearcom codebase spans three layers — a Laravel PHP backend, a React/TypeScript frontend, and a Node.js dev-API — totalling 36 source files and approximately 2,100 combined LOC. No file or class exceeds the 300-LOC threshold, and cyclomatic complexity is well-controlled across all layers (peak ~7). The most significant quality concern is business logic duplication: the reachability-rate calculation is independently implemented in three places (ConnectController.php, RealTimeTestService.php, and dev-api/realtime.js), and the IVR tree-building logic is copy-pasted verbatim across two PHP controllers. A secondary concern is the PHP `extract()` anti-pattern used in `LegacyReportController` directly on `$request->all()`, which simultaneously harms traceability and poses a security boundary risk. Git history is shallow (3 commits, single author), so churn and defect-density signals are low-confidence but show no alarm patterns.

§2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	~7 (StreamController::streamSession)	Good
H2	Large Classes	Largest class/file LOC	<300	300–1000	>1000	222 LOC (ConnectPage.tsx)	Good
H3	Large Functions	Largest function LOC	<50	50–200	>200	~60 LOC (runConnectTest in dev-api/realtime.js)	Moderate
H4	Business Logic Duplication	Duplicated business-rule code %	<5%	5–10%	>10%	~7% (3× reachability calc, 2× buildTree, 2× serializeDoc)	Moderate
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~8% (all H4 patterns plus duplicate query/refetch logic in pages)	Moderate
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	2 max (shallow 3-commit repo)	Good
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	1–2 (server.js, ConnectController.php)	Good
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (ksabai-gl)	Good
H9	PHP `extract()` Anti-Pattern *(additional)*	Unsafe `extract()` calls; ≥1 on user input = Moderate	0	1 internal-only	≥1 user input	1 on `$request->all()` + 1 without EXTR_SKIP	Moderate
H10	Unguarded Error Throw *(additional)*	`throw` in component without Error Boundary	0	≥1	—	1 (LegacyDashboardWidget.tsx:37)	Moderate

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	10	2.50
Code Churn	25%	10	2.50
Defect Density	20%	20	4.00
Class/Function Size	15%	40	6.00
Business Logic Duplication	10%	45	4.50
Developer Ownership Risk	5%	5	0.25
Hotspot Score	**100%**		**20 / 100**

---

§2.5 Actions Required

Hotspot	Action	Rating	Priority
H3 — Large Functions	Extract step-loop runner, state-manager, and diagnostic-writer into separate services in both PHP (RealTimeTestService) and Node.js (realtime.js)	Moderate	Medium
H4 — Business Logic Duplication	Create `ReachabilityService` (PHP) and `reachability.js` (dev-api) as single sources of truth; move `buildTree()` to `IvrTreeBuilder`; consolidate serializers	Moderate	High
H5 — Duplicate Code (general)	Extract `useMonitorQueries` / `useJobQueries` React hooks and a shared `useModuleInvalidation` helper	Moderate	Medium

H9 — PHP `extract()` Anti-Pattern	Replace all `extract()` calls with explicit variable assignment; add PHPStan ban rule; schedule `LegacyDataMapper` removal	Moderate	High
H10 — Unguarded Error Throw	Wrap `LegacyDashboardWidget` in an `<ErrorBoundary>` or replace `throw` with an inline rendered error state	Moderate	Medium

---

§2.6 Expected Outcomes

- **Lower defect risk from rule changes:** Consolidating the reachability calculation into a single service means a threshold change (e.g., 90% → 95%) is a one-line edit that propagates automatically to all consumers, eliminating the current class of "fixed in one place, missed in another" bugs.
- **Faster, safer refactors:** Decomposing the oversized `runDiscoveryTest` / `runConnectTest` methods into focused helpers enables isolated unit testing of the step loop, state management, and diagnostics without real database or sleep-delay dependencies.
- **Elimination of invisible variable injection:** Removing `extract()` from the request path makes data flow explicit and traceable, reducing the surface area for variable-clobbering bugs and simplifying static analysis passes.
- **Improved UI resilience:** Adding an Error Boundary around `LegacyDashboardWidget` prevents transient API errors from crashing the whole dashboard, giving users a degraded-state fallback instead of a blank screen.
- **Cleaner onboarding:** Shared hooks (`useMonitorQueries` , `useJobQueries`) and services (`IvrTreeBuilder` , `ReachabilityService`) give new team members clear, single places to learn domain logic rather than hunting across three files for the same formula.

The full report — including §2.2 Hotspot-by-Hotspot Evidence with code excerpts, §2.3 Churn tables, and §2.4 Mermaid diagrams — has been saved to `docs/discovery/02-code-quality-complexity.md` . The orchestration UI will convert it to PDF automatically.","stop_reason":"end_turn","session_id":"4cf72b34-42cd-4c1d-9a18-a87e20e2c6fc","total_cost_usd":1.2864836999999998,"usage":{"input_tokens":21,"cache_creation_input_tokens":75998,"cache_read_input_tokens":1148309,"output_tokens":31793,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":75998,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":2447,"cache_read_input_tokens":96454,"cache_creation_input_tokens":784,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":784},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":8960,"outputTokens":17,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.009045,"contextWindow":200000,"maxOutputTokens":32000},"cla
sonnet-4-6":{"inputTokens":21,"outputTokens":31793,"cacheReadInputTokens":1148309,"cacheCreationInputTokens":75998,"webSearchRequests":0,"costUSD":1.2774386999999998,"contextWindow":200000,"max
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"0660eede-1adb-4c40-8afa-644b5b3191fc"}